



Московский государственный университет имени М. В. Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра системного программирования

Денисьев Илья Денисович

**Разработка модели файловых систем
и корректных операций над ними
с использованием зависимых типов
для автоматической генерации тестов**

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Научный руководитель:

к.ф.-м.н. Хорошилов Алексей Владимирович

Научный консультант:

Буздалов Денис Викторович

Москва, 2026

Аннотация

Разработка модели файловых систем и корректных операций над ними с использованием зависимых типов для автоматической генерации тестов

Денисьев Илья Денисович

Файловые системы составляют одну из важнейших частей современных операционных систем, являясь большим источником уязвимостей для них. В связи с этим задача обеспечения качества соответствующих драйверов стоит особенно остро. В данной работе представлен метод тестирования файловых систем на основе формальной модели образов файловой системы и семантически корректных операций над ними. Для формализации применён язык программирования с зависимыми типами Idris 2 и библиотека DepTyCheck. Разработанный инструмент `fat32-model` при сравнении с фаззером `syzkaller` показал значительное улучшение по количеству покрытого кода драйвера файловой системы FAT32 операционной системы Linux. Также исследовано обобщение метода на другие файловые системы и создана библиотека `fs-model` для упрощения разработки подобных инструментов для других файловых систем.

Abstract

Development of dependently-typed model for file systems and correct operations on them for automatic test generation

Ilia Denisev

File systems are a critical part of modern operating systems and a major source of vulnerabilities. Thus ensuring the quality of file system drivers is especially important. We present a property-based testing method for file systems which is based on a formal model of file system images and semantically correct operations on them. For specification purposes we use the dependently-typed programming language Idris 2 and the DepTyCheck library. When compared to the `syzkaller` fuzzer, the developed `fat32-model` tool showed a significant improvement in code coverage of the Linux FAT32 file system driver. Generalization of the method to other file systems was also explored, and the `fs-model` library was created to simplify development of similar tools for other file systems.

Содержание

1 Введение	4
2 Постановка задачи	6
3 Обзор существующих решений	6
4 Исследование и построение решения задачи	8
4.1 Моделирование операций в FAT32	8
4.1.1 Основы моделирования	8
4.1.2 Состояние ФС	10
4.1.3 Аргументы операций и индексы	11
4.1.4 Итоговая модель	13
4.1.5 Конвертация в исполняемую программу	14
4.2 Методика оценки результатов	16
4.3 Обобщение на другие ФС	17
5 Результаты	21
5.1 Пример работы	21
5.2 Сравнение с фаззером	22
5.3 Обобщение модели	27
6 Заключение	28
Список литературы	29
Приложение А	31

1 Введение

Одной из главных функций операционных систем является работа с различными файловыми системами. Следовательно, их драйверы являются потенциальными векторами атаки на ядра ОС¹. Ошибки в драйверах ФС² могут привести к аварийным остановкам и уязвимостям, потенциально компрометирующим всю систему. Всё это обеспечивает актуальность тщательного тестирования этих драйверов. Однако эта задача имеет свои сложности — во-первых, в случае недоступности исходного кода драйвера, его приходится рассматривать в виде «чёрного ящика», имея возможность получать и изменять его состояние только посредством определённых системных вызовов. Во-вторых, валидность тех или иных действий с файловой системой зависит от её текущего состояния, которое ещё и изменяется в процессе тестирования. Сложность системы и отсутствие доступа к её полному внутреннему состоянию означает, что ручное написание тестов с полным предугадыванием результата трудоёмко и малоэффективно.

Одним из широко используемых способов тестирования ФС является *fuzzing* — автоматическая генерация случайных входных данных с целью спровоцировать ошибки или падения программы. В контексте файловых систем под программой понимается драйвер, а под входными данными — начальные образы ФС и последовательности операций над ними. Генерация обычно происходит за счёт мутации существующих, т. е. ранее сгенерированных или изначально заданных, входных данных. При этом направление мутации часто выбирается на основании какой-то обратной связи, например, степени покрытия кода. Полученные данные вовсе не обязательно являются валидными — это позволяет проверить, как программа справляется с некорректными данными и обработкой ошибок, но может привести к потенциально меньшему покрытию более «глубоких» путей в коде, т. к. невалидные данные будут отсеиваться на более ранних этапах работы программы. Из-за этого в большинстве применений фаззеров не проводится полноценное функциональное тестирование системы.

Другим способом является *property-based*³ тестирование, которое заключается в проверке определённых свойств тестируемой системы, выраженных как общие зависимости результата работы системы от заданных входных данных. Это противопоставляется проверке соответствия некоторому эталону, что особенно актуально для тестирования файловых систем, где итоговое состояние часто не определяется одно-

¹операционных систем

²файловых систем

³основанное на свойствах

значно. Свойства задаются *спецификацией*, на основе которой случайным образом генерируются входные данные. Примерами свойств для нашей задачи тестирования ФС являются:

- отсутствие фатальных ошибок драйвера и ядра ОС,
- успешное монтирование образа,
- соответствие результата операции чтения и ожидаемого содержимого файла,
- правильный код ошибки при попытке записи в файл только для чтения.

Для решения обозначенных проблем в данной работе представлен метод property-based тестирования файловых систем на основе формальной модели образов файловой системы и семантически корректных операций над ними. Модель реализована на *Idris 2* [1–3] — чистом функциональном языке программирования общего назначения с *зависимыми типами* [4]. Для генерации используется разрабатываемая в ИСП РАН библиотека *DepTyCheck* [5,6], которая позволяет *деривировать*⁴ генераторы значений типа по его описанию на этапе компиляции, благодаря встроенному в *Idris* механизму метапрограммирования *elaborator reflection* [7]. На основе предыдущей работы [8] по генерации корректных бинарных образов ФС FAT32 в данной работе разработан инструмент *fat32-model* [9] для генерации программ на языке Си, выполняющих корректные операции над сгенерированным образом и проверяющих результаты их работы, производя таким образом функциональное тестирование драйвера FAT32 ядра ОС Linux. В рамках работы также рассмотрено обобщение метода на другие файловые системы и разработана библиотека *fs-model* [10] для упрощённой реализации инструментов, позволяющих генерировать корректные операции для тестирования других ФС.

⁴автоматически выводить

2 Постановка задачи

Целью работы является разработка метода property-based тестирования реализаций файловых систем на основе формальной модели корректных образов ФС и семантически корректных операций над ними с помощью зависимых типов.

Для достижения цели в рамках работы необходимо решить следующие задачи:

- создать формальную спецификацию с использованием зависимых типов для семантически корректных операций над файловой системой FAT32;
- на её основе разработать генератор программ на языке Си, осуществляющих эти операции с помощью системных вызовов ОС Linux и проверяющих их результат;
- провести сравнение с существующими инструментами обеспечения качества реализаций файловых систем;
- обобщить полученную спецификацию для добавления поддержки других файловых систем.

3 Обзор существующих решений

Для фаззинга ядра Linux широко применяется *syzkaller*[11] — coverage-guided⁵ фаззер, который генерирует последовательности системных вызовов, постепенно мутируя их из начального корпуса, и запускает их в виртуальных машинах, собирая покрытие. Syzkaller применяется в том числе и для тестирования драйверов файловых систем, но не заточен под него — образы и операции мутируются таким же образом, как и любые другие аргументы системных вызовов. Это приводит к тому, что чаще всего они оказываются некорректными. Изначальные образы для мутации получаются[12] с помощью *mkfs.fat* и обычного копирования файлов, что также потенциально ограничивает пространство получаемых образов. Например, во многих файловых системах файлы могут быть фрагментированы, но при создании образов вышеописанным способом драйвер обычно записывает фрагменты последовательно, оставляя в итоге непокрытыми многие тестовые случаи. Генерируемые же операции никак не опираются на текущее состояние ФС, потенциально снижая глубину тестирования, а значения результатов их выполнения никак не проверяются.

Исследователями из технологического института Джорджии, пытавшимися увеличить эффективность фаззинга конкретно файловых систем, был разработан инструмент *Janus*[13]. Три основных преимущества перед *syzkaller*:

- образы мутируются с учётом их структуры, при надобности пересчитываются контрольные суммы, мутируются в основном метаданные (таким образом, работа идёт по большей части с корректными образами);

⁵управляемый покрытием

- Janus поддерживает предполагаемое состояние ФС в памяти в процессе фаззинга, что позволяет генерировать только корректные в данный момент действия;
- вместо виртуальных машин используется LKL (Linux Kernel Library)[14] для улучшения воспроизводимости падений.

Однако Janus всё ещё не занимается генерацией образов с нуля, а мутирует заданные начальные образы, и если они недостаточно разнообразные (как и в случае с `syzkaller`), то могут быть такие же проблемы с потенциальным ограничением пространства рассматриваемых образов. Значения результатов выполнения операций также не проверяются, а отслеживание состояния ФС ограничено — в пределах одной программы системные вызовы могут мутироваться только таким образом, чтобы потенциально не влиять на валидность дальнейших операций.

Инструменты интерактивного доказательства теорем (в число которых входит и `Idris`) также не в первый раз используются для обеспечения качества драйверов файловых систем — существует формализация [15] FAT32 в системе ACL2. В рамках этой работы были также разработаны формально верифицированные реализации некоторых системных вызовов для FAT32, но не метод генерации образов, соответствующих модели. Тем не менее, рассматриваемая в статье методология инкрементального построения модели путём постепенного понижения уровня абстракции хорошо переносится и на `Idris`, и построение `fat32-model` происходило похожим образом.

4 Исследование и построение решения задачи

Общая схема работы представлена на рис. 1. Зелёным выделено то, что было сделано в рамках курсовой работы[8], синим — в рамках текущей работы.

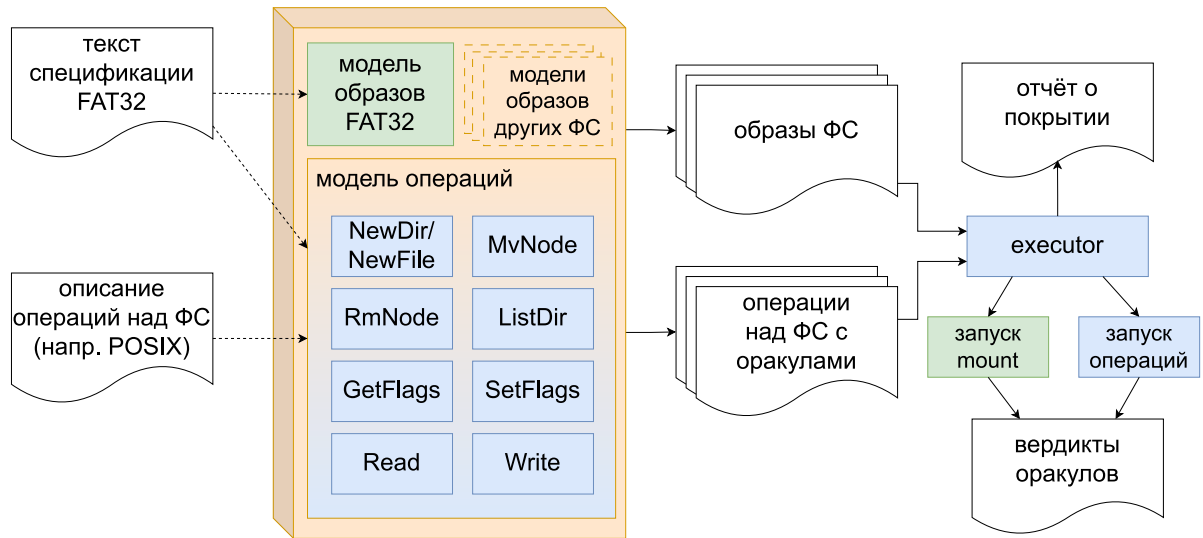


Рис. 1. Общая схема работы

4.1 Моделирование операций в FAT32

Задача состоит в том, чтобы, имея результат генератора образов как начальное состояние ФС FAT32, сгенерировать последовательность корректных операций для данного состояния. При этом важно, что каждая операция может менять это состояние. Таким образом, операцию можно рассматривать как состоящую из двух основных частей:

- аргументы операции;
- функция перехода состояния ФС, зависящая от аргументов операции.

Внутреннее представление последовательности операций в таком случае является связным списком, где каждый новый «хвост» получает получившееся после операции в «голове» состояние в качестве изначального.

4.1.1 Основы моделирования

Для демонстрации основных принципов моделирования операций на языке Idris мы рассмотрим небольшой пример игрушечной ФС. Пусть её состояние выглядит как вектор из k байт, а единственная допустимая операция — чтение байта под номером i .

```

1 data NodeOps : Vect k Bits8 → Type where
2   ||| 1. Пустой список операций (аналогично nil в связном списке)
3   Nop : NodeOps fs
4   ||| 2. Операция чтения – аргумент и продолжение списка (аналогично cons)
5   Read : (i : Nat) →
6           (cont : NodeOps fs) →
7           NodeOps fs

```

Листинг 1. Тип операций над игрушечной ФС

На лист. 1 видно, что у конструктора `Read` два аргумента: `i` — номер интересующего нас байта, выраженный как `Nat`, т. е. натуральное (целое неотрицательное) число, и `cont` — продолжение списка операций. Заметим, что данная операция не меняет состояние ФС, поэтому в текущем списке и в продолжении один и тот же параметр `fs`.

Но у такого типа есть критический недостаток — к примеру, `Read 4 Nop` является валидным значением типа `NodeOps [0xbe, 0xef]`, что соответствует некорректной операции получения байта с индексом 4 в векторе размера 2. Для нашей задачи критически важно генерировать только корректные операции, поэтому тип последовательности операций должен быть такой, чтобы все его допустимые значения соответствовали корректным операциям и только им.

К счастью, на языке `Idris` можно выразить тип натуральных чисел, строго меньших заданного числа n — `Fin n`, поэтому мы можем уточнить тип, ограничив аргумент `i` только валидными индексами:

```

1 data NodeOps : Vect k Bits8 → Type where
2   Nop : NodeOps fs
3   Read : (i : Fin k) →
4           (cont : NodeOps {k} fs) →
5           NodeOps fs

```

> Здесь мы явно указываем, что k в типе аргумента `i` соответствует размеру вектора `fs`.

Листинг 2. Тип операций над игрушечной ФС с ограниченным индексом

Для демонстрации функций перехода добавим операцию `Write`, которая будет изменять байт в векторе по индексу i на байт a .

```
1 data NodeOps : Vect k Bits8 → Type where
2   Nop : NodeOps fs
3   Read : (i : Fin k) →
4         (cont : NodeOps {k} fs) →
5         NodeOps fs
6   Write : (a : Bits8) →
7          (i : Fin k) →
8          (cont : NodeOps $ replaceAt i a fs) →
          > В хвосте списка операций параметр уже не fs, т. к. после этой
операции состояние ФС меняется — новым состоянием становится результат
выполнения функции replaceAt.
9   NodeOps fs
```

Листинг 3. Тип операций над игрушечной ФС с операцией `Write`

Здесь `Write` также позволяет выразить только корректные операции записи, т. к. индекс i заведомо находится в пределах вектора. Кроме того, все следующие операции будут работать уже с изменённым состоянием, где произошла запись. Это позволяет нам проверять результаты операций чтения, генерируя оракулы на основе текущего состояния системы.

Таким образом, мощная система типов языка Idris позволяет описывать структуры данных со сложными инвариантами, благодаря чему наша модель может быть спроектирована так, чтобы допускать исключительно корректные значения.

4.1.2 Состояние ФС

Для начала стоит определиться, как будет моделироваться состояние системы. Мы уже имеем внутреннюю структуру, описывающую готовую ФС, которая затем (вместе с некоторыми вспомогательными структурами) детерминированным образом преобразуется в бинарный образ — именно её и было решено использовать в качестве модели состояния. Преимуществом такого подхода является то, что не нужно описывать дополнительных структур и реализовывать преобразования между ними — сгенерированная модель ФС может сразу использоваться в качестве входного параметра генератора операций. Недостатком же является наличие нерелевантных для генерации операций деталей в структуре: если одни инварианты нам нужны (например, уникальность имён файлов в пределах одной директории), то такая информация, как количество занимаемых кластеров⁶, полезна только для генерации бинарных образов, а при моделировании операций только затрудняет реализацию.

⁶кластерами в FAT32 называются блоки фиксированного размера, на которые разбивается каждый файл

Дополнительную сложность при моделировании состояния ФС добавляет то, что не у всех операций переходы состояния определяются однозначно — мы можем только предсказывать ожидаемое состояние на основании спецификации[16], которая описывает не всё. Например, у файлов в FAT32 есть атрибут «архивный», который используется некоторыми утилитами резервного копирования, чтобы пометить файлы, которые уже были архивированы. Согласно спецификации, драйвер должен устанавливать этот атрибут у файла при его создании, переименовании и записи в него. Но поведение драйвера касательно директорий и при перемещении файлов явно не специфицировано и, как показали эксперименты, отличается в разных реализациях. По этой причине наши оракулы не проверяют значение этого атрибута при выполнении операции получения атрибутов файла. Возможной альтернативой в данном случае является добавление в модель третьего возможного значения атрибута — «неизвестно», которое будет устанавливаться после операций, которые могут изменить этот атрибут неспецифицированным образом, и перезаписываться после операций, которые устанавливают атрибут в заведомо известное положение. Проверяться значение этого атрибута будет, соответственно, только если оно не «неизвестно».

4.1.3 Аргументы операций и индексы

Далее следует определиться с моделированием аргументов операций. Поскольку мы хотим генерировать только корректные операции, то аргументы должны моделироваться так, чтобы все допустимые системой типов значения были валидными. Мы уже имеем инфраструктуру для проверки уникальности имён в директории и таких предикатов, как «сдвиг внутри файла при чтении не выходит за его границы». Однако главный тип аргумента, присутствующий в почти любой операции — это путь к файлу. Соответственно, нам нужно уметь моделировать валидные пути для заданного состояния ФС.

В стандартной библиотеке Idris есть тип `Data.List.Elem`, соответствующий утверждению, что заданный элемент принадлежит списку. Доказательство этого факта разбивается на два случая, как видно на лист. 4:

```
1 data Elem : a → List a → Type where
2   ||| 1. Элемент лежит в голове списка (база рекурсии)
3   Here  : Elem x (x :: xs)
4   ||| 2. Элемент лежит в хвосте списка (переход рекурсии)
5   There : Elem x xs → Elem x (y :: xs)
```

Листинг 4. Описание типа `Data.List.Elem` из стандартной библиотеки

Мы можем применить похожую идею для индексации внутри директории, используя вместо `List` наш внутренний тип для хранения содержимого директории.

При этом здесь мы не хотим задавать какое-то ожидаемое значение элемента заранее, тип используется исключительно как всегда валидный «указатель» в директории:

```
1 data ShallowIndexIn : HSnocVectMaybeNode cfg k ars prs → Type where
2   Here : ShallowIndexIn (xs :< Just x)
3   There : ShallowIndexIn xs → ShallowIndexIn (xs :< x)
```

Листинг 5. Описание «неглубокого» индекса по директории

Описанный выше индекс является «неглубоким» в том смысле, что для своей заданной директории он может указывать только на кого-то из её непосредственных потомков. Для описания произвольных путей от корня файловой системы нам нужен «глубокий» индекс, параметризующийся не списком, а целым поддеревом:

```
1 data IndexIn : Node cfg ar fs → RootLabel → IndexDirLabel → Type where
2   ||| 1. Указатель на текущий узел (корень текущего поддерева)
3   HereFile : IndexIn (File meta blob)      NotRoot FileI
4   HereDir  : IndexIn (Dir meta entries names) NotRoot DirI
5   HereRoot : IndexIn (Root entries names)   IsRoot  DirI
6   ||| 2. Указатель внутри поддерева (взаимная рекурсия)
7   ThereDir : DIndexIn xs dirl → IndexIn (Dir meta xs names) IsRoot dirl
8   ThereRoot : DIndexIn xs dirl → IndexIn (Root xs names)   IsRoot dirl
```

Листинг 6. Описание «глубокого» индекса по узлу

Заметим, что в `ThereDir` и `ThereRoot` содержится глубокий индекс по содержанию соответствующей директории (см. лист. 7), который устроен как неглубокий индекс, описанный выше, но вместо рекурсивной ссылки на себя он содержит на индекс по поддереву. Т. е. получившаяся структура является взаимно-рекурсивной.

```
1 data DIndexIn : HSnocVectMaybeNode cfg k ars prs → IndexDirLabel → Type
   where
2   Here : IndexIn x NotRoot dirl → EntriesIndexIn (xs :< Just x) dirl
3   There : IndexIn xs dirl      → EntriesIndexIn (xs :< x) dirl
```

Листинг 7. Описание «глубокого» индекса по директории

Также заметим, что индексы дополнительно имеют два параметра:

- указывает ли индекс на корневой узел;
- указывает ли индекс на файл или директорию (в т. ч. корневую).

Это позволяет ограничивать виды файлов в аргументах некоторых операций, например, создать файл можно только внутри директории.

4.1.4 Итоговая модель

Отрывок структуры, описывающей последовательность операций, представлен на лист. 8, а полный список реализованных операций — в табл. 1.

```
1 data NodeOps : (cfg : NodeCfg) → (root : Node cfg ar Rootful) → Type where
2   ||| Пустой список операций
3   Nop : NodeOps cfg root
4   ||| Операция получения атрибутов файла
5   GetFlags :
6     ||| Указатель на файл
7     (idx : IndexIn root rootl dirI) →
8     ||| Хвост списка операций
9     (cont : NodeOps cfg root) →
10  NodeOps cfg root
    > Заметим, что операция получения атрибутов не меняет состояние ФС,
    поэтому хвост списка параметризуется тем же состоянием root, что и
    изначальный список.
11  ||| Операция перемещения/переименования файла
12  MvNode :
13    ||| Указатель на директорию-источник
14    (idx : IndexIn root rootl DirI) →
15    ||| Указатель на файл внутри директории-источника
16    (sidx :
17      ShallowIndexIn $ fst $ snd $ snd $ snd $ getContsByDirIdx root idx) →
18    ||| Указатель на директорию назначения
19    (idx2 : IndexIn (snd $ rmNode cfg root idx sidx) rootl DirI) →
20    ||| Новое имя файла
21    (name : Filename) →
22    ||| Доказательство уникальности нового имени в директории назначения
23    (nameprf : NameIsNew
24      (snd $ snd $ snd $ snd $ getContsByDirIdx
25        (snd $ rmNode cfg root idx sidx)
26        idx2)
27      (Just name)) →
28    ||| Хвост списка операций
29    (cont :
30      NodeOps cfg $ snd $ mvNode cfg root idx sidx idx2 name nameprf) →
31  NodeOps cfg root
    > Здесь же операция меняет состояние, поэтому с помощью функции mvNode
    вычисляется новое состояние, которым затем параметризуется хвост.
32  -- остальные операции...
```

Листинг 8. Описание последовательности операций

название	описание
GetFlags	получение атрибутов файла
SetFlags	установка новых атрибутов файла
NewDir	создание новой пустой директории
NewFile	создание нового пустого обычного файла
RmNode	удаление файла
MvNode	перемещение/переименование файла
LsDir	перечисление содержимого директории
Read	чтение обычного файла
Write	запись в обычный файл

Таблица 1. Список реализованных операций над ФС FAT32

4.1.5 Конвертация в исполняемую программу

Наконец, сгенерированную структуру необходимо преобразовать в исполняемую программу, которая будет производить полученные операции над файловой системой и проверять их результат. В рамках данной работы была реализована конвертация в программу на языке Си с использованием системных вызовов операционной системы Linux. Тем не менее, описанная архитектура генератора позволяет при желании добавить и другие форматы вывода операций, поскольку генерируемая структура абстрагирована от деталей конкретного формата.

```

1 printC0ps : Nat → Nat →
2     (node : Node cfg ar Rootful) → NodeOps cfg node → List String
3 -- другие операции...
4 printC0ps i len root (MvNode idx sidx idx2 dstname nameprf cont) = let
5     ||| Индексы преобразуются в Unix пути
6     srcdir = index2UnixPath root idx
7     srcname = indexPair2Name root idx sidx
8     dstdir = index2UnixPath (snd $ rmNode _ _ _ sidx) idx2
9     in #"""
10         {
11             puts("Test \#{show i}/\#{show len}: "
12                 "MvNode from \#{srcdir}\#{srcname} to \#{dstdir}\#{dstname}");
13             errno = 0;
14             int srcfd = openat(rootfd, "\#{srcdir}", O_PATH | O_DIRECTORY);
15             panic_on(srcfd < 0);
16             int dstfd = openat(rootfd, "\#{dstdir}", O_PATH | O_DIRECTORY);
17             panic_on(dstfd < 0);
18             int res = renameat2(srcfd, "\#{srcname}",
19                                 dstfd, "\#{dstname}", RENAME_NOREPLACE);
20             panic_on(res < 0);
21             res = close(srcfd);
22             panic_on(res < 0);
23             res = close(dstfd);
24             panic_on(res < 0);
25         }
26         """"# :: printC0ps (i + 1) len _ cont
27 -- другие операции...

```

Листинг 9. Реализация преобразования операции на примере MvNode

Деталью реализации является отсутствие в описанной структуре ожидаемых значений результатов операций, которые должны проверяться оракулами, т. к. они однозначно восстанавливаются из описания операций во время конвертации в программу. Например, для известного сдвига в файле и длины вычисляется ожидаемый результат операции `Read`. При наличии нескольких поддерживаемых форматов вывода может быть полезно определить дополнительную промежуточную структуру с результатами операций, в которую сгенерированная последовательность будет преобразовываться до конвертации в программу, чтобы не дублировать вычисление результатов. Ещё одним возможным улучшением может быть недетерминированное преобразование в программу, при котором для каждой операции будет случайно выбираться произвольная реализация из возможных. Тем не менее, оба этих улучшения выходят за рамки данной работы.

4.2 Методика оценки результатов

Как и в прошлой работе[8], сравнение разработанного инструмента будет проводиться с фаззером `syzkaller`, а основной метрикой будет покрытие исходного кода драйвера FAT32 ядра Linux. Помимо сравнения обычного фаззера и разработанного инструмента, можно также сравнить покрытие с фаззером, среди начальных данных которого будут образы, сгенерированные нашим инструментом.

Поскольку разработанный инструмент не имеет встроенной инфраструктуры для сбора информации о покрытии, её нужно каким-то образом собирать отдельно. `Syzkaller` использует для этих целей механизм ядра Linux `KCOV`[17], предназначенный специально для фаззинга — он достаточно гранулярен, позволяя собирать информацию о покрытии отдельных системных вызовов, и достаточно быстр, используя общий буфер в памяти для получения информации от ядра. Однако если получаемой с помощью `KCOV` информации хватает для работы фаззера (который принимает на её основе решения о мутации тех или иных данных), то для получения интерпретируемых отчётов она не подходит — бóльшая часть строк оказывается без какой-либо информации о покрытии, а та информация, что есть, как правило неконсистентна. Это очень затрудняет анализ результатов (как количественный, так и качественный). Другой поддерживаемый в ядре Linux механизм, `gcov`[18], позволяет получать более интерпретируемые отчёты, но из-за оптимизаций компилятора соответствие между собранным покрытием и исходным кодом тоже оказывается неточным.

Исследователи из Иллинойского университета и компании Boeing попытались решить эту проблему[19], предложив использовать механизм `Source-based Code Coverage`[20] (покрытие, основанное на исходном коде) компилятора `clang` и сопутствующего инструмента `llvm-cov` для сбора покрытия кода ядра Linux. Преимущество этого механизма состоит в том, что компилятор учитывает оптимизации и прочие преобразования при инструментировании кода, позволяя получить очень точную информацию о покрытии, как видно на рис. 2.

1172	455	if (opt == -ENOPARAM) {	61	1172	if (opt == -ENOPARAM) {
1173	354	if (is_vfat)		1173	if (is_vfat)
1174	354	opt = fs_parse(fc, vfat_param_spec, param, &result);		1174	opt = fs_parse(fc, vfat_param_spec, param, &result);
1175	0	else		1175	else
1176	0	opt = fs_parse(fc, msdos_param_spec, param, &result);		1176	opt = fs_parse(fc, msdos_param_spec, param, &result);
1177	354	}		1177	}
1178				1178	
1179	455	if (opt < 0)		1179	if (opt < 0)
1180	100	return opt;		1180	return opt;
1181				1181	
1182	355	switch (opt) {		1182	switch (opt) {
1183	2	case Opt_check:		1183	case Opt_check:
1184	2	opts->name_check = result.uint_32;		1184	opts->name_check = result.uint_32;
1185	2	break;		1185	break;
1186	1	case Opt_usefree:		1186	case Opt_usefree:
1187	1	opts->usefree = 1;		1187	opts->usefree = 1;
1188	1	break;		1188	break;
1189	1	case Opt_nocase:		1189	case Opt_nocase:
1190	1	if (!is_vfat)		1190	if (!is_vfat)
1191	0	opts->nocase = 1;		1191	opts->nocase = 1;
1192	1	else {		1192	else {
1193		/* for backward compatibility */		1193	/* for backward compatibility */
1194	1	opts->shortname = VFAT_SFN_DISPLAY_WIN95		1194	opts->shortname = VFAT_SFN_DISPLAY_WIN95
1195	1	VFAT_SFN_CREATE_WIN95;		1195	VFAT_SFN_CREATE_WIN95;
1196	1	}		1196	}

Рис. 2. Сравнение отчётов llvm-cov (слева) и syzkaller/KCOV (справа)

На момент написания статьи поддержка этого механизма ещё не была официально включена в ядро Linux, но нужные изменения можно применить самостоятельно.

Поскольку мы больше не используем KCOV для анализа покрытия кода, собирать информацию о работе фаззера также придётся самостоятельно. В процессе своей работы фаззер собирает так называемый «корпус» — набор программ в специальном формате, достаточный для покрытия всего кода, который удалось затронуть за время фаззинга. Запустив все программы в этом корпусе внутри виртуальной машины с нужной модификацией ядра, мы можем получить отчёт о покрытии, достигнутом фаззером, в нужном нам формате.

4.3 Обобщение на другие ФС

Файловые системы, как правило, сильно отличаются по устройству своего формата на диске, поэтому когда основная задача состоит в генерации образов, обобщение подхода на разные ФС довольно затруднительно. Однако когда речь идёт о генерации последовательностей операций, обобщение становится значительно более осуществимым, ведь операционные системы обычно предоставляют один и тот же базовый интерфейс для взаимодействия со всеми ФС. Действительно, практически все ФС имеют древовидную структуру, файлы и директории, базовые операции

(создание/удаление файлов и директорий, чтение/запись в файл). Тем не менее, есть и немало отличий, затрудняющих такое обобщение, например:

- разные ограничения на имена файлов — имена могут быть чувствительны и не чувствительны к регистру, могут допускать разные наборы символов, требовать определённые кодировки, иметь разные ограничения на длину и т. д.;
- разные модели прав доступа — некоторые поддерживают модель прав доступа Unix, некоторые могут дополнительно поддерживать какие-то формы ACL (списков управления доступом), а некоторые (как, например, FAT32) вообще могут иметь лишь атрибут «только для чтения»;
- различные метаданные, такие как, например, разные виды временных меток и их формат;
- поддержка других дополнительных функций: например, расширенных атрибутов (xattr);
- наличие специфичных для конкретной ФС операций (которые в ОС Linux обычно осуществляются при помощи системного вызова `ioctl`).

В рамках данной работы мы не будем рассматривать такие типы файлов, как символичные ссылки, устройства, сокеты и др. Наша модель файловой системы будет представлять из себя дерево, где каждый узел имеет один из трёх типов: корневая директория (не имеет предка), директория и обычный файл (не может иметь потомков).

```
1 data FsNode : (mtty : RootLabel → Type) → (fty : Type) →
2   RootLabel → Type where
3   File : (meta : mtty NotRoot) →
4     (blob : SnocVect k Bits8) →
5     FsNode mtty fty NotRoot
6   Dir  : (meta : mtty NotRoot) →
7     (names : UniqNames prs fty) →
8     (entries : FsEntries prs mtty fty) →
9     FsNode mtty fty NotRoot
10  Root : (meta : mtty IsRoot) →
11    (names : UniqNames prs fty) →
12    (entries : FsEntries prs mtty fty) →
13    FsNode mtty fty IsRoot
```

Листинг 10. Описание обобщённой модели ФС

Тип узла параметризуется типом метаданных и типом имени файла. Т. к. метаданные корневого узла — это по сути метаданные всей ФС, тип метаданных зависит от того, является ли узел корневым (поэтому мы выделяем корень как отдельный вид узла). Некоторые операции также могут взаимодействовать с метаданными определённым образом (например, для валидности операции записи в файл

нужно, чтобы файл был доступен для записи), поэтому тип метаданных может реализовывать определённые интерфейсы. В любой директории есть ограничение на уникальность имён файлов, поэтому значения заданного типа имён нужно уметь проверять на равенство. Также практически для всех операций над ФС всё также нужен способ указывать на существующие файлы и директории в дереве, поэтому аналогичным образом мы определяем для этой структуры индексы.

Для того, чтобы иметь возможность использовать пересекающиеся множества операций для разных файловых систем без дублирования кода, нам сначала нужно абстрагировать тип одной операции от типа последовательности операций.

```

1 0 FsOpsType : (RootLabel → Type) → Type → Type
2 FsOpsType mty fty = FsNode mty fty IsRoot → FsNode mty fty IsRoot → Type
3
4 data PosixOps : FsOpsType mty fty
5 data XattrOps : Xattr mty => FsOpsType mty fty
6 data FAT32Ops : FsOpsType FAT32Meta FAT32Filename

```

Листинг 11. Обобщённый параметризованный тип операции

Отметим, что здесь параметрами являются как начальное, так и конечное состояние ФС — это позволит обеспечить корректность переходов состояния в последовательности.

Множества операций можно склеивать, позволяя легко использовать их для разных ФС.

```

1 (|+|) : FsOpsType mty fty → FsOpsType mty fty → FsOpsType mty fty
2 (|+|) l r pred f = Either (l pred f) (r pred f)
3
4 JoinedOps = PosixOps |+| XattrOps

```

Листинг 12. Склеивание типов операций

Наконец, из операций можно составлять последовательности.

```

1 data FsOpsSeq : FsOpsType mty fty → FsNode mty fty IsRoot → Type where
2 Nil : {0 no : FsOpsType mty fty} →
3     FsOpsSeq no st
4 (::) : {0 no : FsOpsType mty fty} →
5     (op : no f t) →
6     (cont : FsOpsSeq no t) →
7     FsOpsSeq no f

```

Листинг 13. Последовательности операций

Заметим, что тип последовательностей уже параметризуется только начальным состоянием. Это связано с тем, что для всей последовательности мы хотим, чтобы начальное состояние соответствовало сгенерированному образу, в то время как конечное состояние может быть любым.

В данной работе мы не ставим целью исчерпывающе описать все операции какого-то существующего интерфейса. Вместо этого, опираясь на интерфейс POSIX, мы описываем подмножество абстрактных операций, которые достаточно распространены и которых достаточно для выражения существующей модели ФС FAT32 через обобщённую. Список реализованных обобщённых операций `PosixOps` приведён в табл. 2.

название	описание
<code>NewDir</code>	создание новой пустой директории
<code>NewFile</code>	создание нового пустого обычного файла
<code>RmNode</code>	удаление файла
<code>MvNode</code>	перемещение/переименование файла
<code>LsDir</code>	перечисление содержимого директории
<code>Read</code>	чтение обычного файла
<code>Write</code>	запись в обычный файл

Таблица 2. Список реализованных обобщённых POSIX-подобных операций

Таким образом, для добавления поддержки новой ФС нужны:

- тип метаданных, для которого должны быть реализованы интерфейсы для соответствующих операций;
- тип имён файлов, значения которого нужно уметь проверять на равенство и печатать;
- тип операций, для которого объединяются существующие типы операций и, возможно, добавляются свои; при этом для новых операций понадобится определить, как они конвертируются в тесты поддерживаемых форматов;
- дополнительные типы и функции для генерации бинарных образов.

Также важно отметить, что многие типы, описанные выше, параметризуются другими типами. Без этого обобщить модель было бы затруднительно, но на момент написания статьи поддержка таких типов в `DerTyCheck` ещё находится в разработке. По этой причине в рамках данной работы мы ограничиваемся только лишь описанием обобщённой модели без применения её для генерации тестов.

5 Результаты

5.1 Пример работы

Разработанный инструмент[9] принимает следующие параметры:

-c <size>	--cluster-size=<size>	cluster size in bytes
-1 <fuel1>	--fuel1=<fuel1>	fuel for the Node generator
-2 <fuel2>	--fuel2=<fuel2>	fuel for the NameTree generator
-s <seed>	--seed=<seed>	seed
-m <minclust>	--minclust=<minclust>	min amount of data clusters
-b <blob-limit>	--blob-limit=<blob-limit>	max image generation blob size
-w <write-limit>	--write-limit=<write-limit>	max write operation blob size
-N	--print-node	print the generated Node
-F	--print-fs	pretty-print the FS tree
-C	--print-cmap	print the generated cluster map
-I	--image-only	generate only the image
-h	--help	print usage information
-o <output>	--output=<output>	output image filename

Пример запуска:

```
pack run fat32-model -s 96769337 -c 512 -1 27 -b 2048 -2 256 -w 196
```

Генерация занимает примерно 6 мин и менее 1 ГиБ оперативной памяти на сервере с Intel Xeon E5-2690 v4 (2.60 ГГц, 56 потоков). Получившийся образ содержит 1351 директорию и 624 файла, а программа — 224 операции.

```
1 puts("Test 220/224: LsDir on ")
2  ". /4BMP)UP/(X`2BCHT/1.7-U/422.R/W0~.XB/XUW.}B/JMN8$Z.H7}/0I'!1.8)U/");
3 errno = 0;
4 const char *golden[] = {"4~F.P", "L.^L", };
5 struct dirent **namelist;
6 int res = scandirat(rootfd,
7  ". /4BMP)UP/(X`2BCHT/1.7-U/422.R/W0~.XB/XUW.}B/JMN8$Z.H7}/0I'!1.8)U/",
8  &namelist, NULL, direntcmp);
9 panic_on(res < 0);
10 assert(res - 2 == sizeof(golden) / sizeof(*golden));
11 int j = 0;
12 for (int i = 0; i < res; ++i) {
13     if (strcmp(".", namelist[i]→d_name) != 0
14         && strcmp("..", namelist[i]→d_name) != 0) {
15         assert(!strcmp(golden[j], namelist[i]→d_name));
16         ++j;
17     }
18     free(namelist[i]);
19 }
20 free(namelist);
```

Рис. 3. Пример сгенерированной операции

5.2 Сравнение с фаззером

В качестве эксперимента с помощью `fat32-model` было сгенерировано 64 пары из образа и программы к нему, в каждой программе в среднем примерно по 200 операций. Генерация проводилась на 50 потоках вышеупомянутого сервера и заняла примерно 20 минут. Была запущена виртуальная машина с ядром Linux версии 6.19.6, к которой были применены вышеупомянутые модификации для поддержки сбора покрытия с помощью `lvm-cov`. Образы и программы были перенесены на виртуальную машину — 64 цикла монтирования и запуска заняли примерно 10 минут. Результат запуска виден на табл. 3, где для каждого файла исходного кода драйвера FAT32 указаны процент покрытия по функциям (Function Coverage), по строкам (Line Coverage) и процент по критерию MC/DC⁷.

Filename	Function Coverage	Line Coverage	MC/DC
cache.c	88.89% (16/18)	76.34% (200/262)	16.67% (1/6)
dir.c	71.88% (23/32)	49.05% (489/997)	18.75% (12/64)
fat.h	78.26% (18/23)	65.52% (95/145)	50.00% (2/4)
fatent.c	61.76% (21/34)	48.91% (293/599)	0.00% (0/21)
file.c	66.67% (10/15)	47.92% (173/361)	0.00% (0/19)
inode.c	72.73% (40/55)	53.83% (674/1252)	2.63% (1/38)
misc.c	76.92% (10/13)	57.42% (120/209)	0.00% (0/10)
namei_msdos.c	0.00% (0/19)	0.00% (0/507)	0.00% (0/49)
namei_vfat.c	76.74% (33/43)	57.83% (480/830)	5.41% (2/37)
nfs.c	0.00% (0/11)	0.00% (0/185)	0.00% (0/9)
Totals	65.02% (171/263)	47.20% (2524/5347)	7.00% (18/257)

Таблица 3. Статистика покрытия драйвера FAT32 после работы `fat32-model`

Для сравнения было запущено три экземпляра фаззера `syzkaller`, отличающихся набором начальных образов для мутации — помимо встроенных 24 образов, созданных с помощью `mkfs.fat`, было отдельно сгенерировано ещё столько же образов с помощью `fat32-model`.

название	встроенные	сгенерированные
syz-mkfs	+	
syz-idr		+
syz-both	+	+

Таблица 4. Запущенные экземпляры фаззера

Эксперимент производился на ревизии фаззера `0b6ab7ec`, к которой были применены следующие модификации[21]:

⁷Modified Condition/Decision Coverage (модифицированный метод покрытия по веткам/условиям) — критерий покрытия, широко используемый в авиационной индустрии

1. максимальный размер образа увеличен с 132 до 300 МиБ, что нужно для корректной поддержки образов, сгенерированных `fat32-model`;
2. убраны образы FAT12 и FAT16 из изначальных, поскольку в рамках этой работы мы рассматриваем только FAT32;
3. отключена минимизация корпуса при записи его на диск, т. к. она приводит к ложному уменьшению измеряемого покрытия.

На каждый экземпляр фаззера выделено по 3 виртуальной машины с 1 ЦП, 4 ГиБ памяти и 2 процессами фаззера на каждую. Все виртуальные машины используются только для фаззинга, воспроизведение падений во время эксперимента было выключено. Системные вызовы при фаззинге ограничены так, чтобы сфокусироваться на тестировании ФС, при этом учтены также специфичные для FAT32 операции, как видно на лист. 14.

```
exit, syz_mount_image$vfat, umount2, open, creat, close, read, pread64, readv,
preadv, preadv2, write, pwrite64, writev, pwritev, pwritev2, lseek, sendfile,
stat, lstat, fstat, newfstatat, statx, unlink, unlinkat, mkdir, mkdirat, rmdir,
rename, renameat, renameat2, truncate, ftruncate, chdir, fchdir, fsync,
fdatasync, sync, syncfs, getdents, getdents64, statfs, fstatfs,
ioctl$FAT_IOCTL_GET_ATTRIBUTES, ioctl$FAT_IOCTL_SET_ATTRIBUTES,
ioctl$FAT_IOCTL_GET_VOLUME_ID
```

Листинг 14. Список разрешённых для фаззинга системных вызовов

Замеры покрытия фаззинга производились три раза: после 30 мин, 1 дня и 7 дней непрерывной работы фаззера соответственно. Первый замер производился через 30 мин, т. к. именно столько времени было суммарно потрачено на тестирование с помощью `fat32-model`. Это позволяет сравнить результативность методов за схожий промежуток времени. Прошлый опыт работы с `syzkaller` показал, что для получения достаточно значимых результатов обычно нужно несколько дней непрерывного фаззинга, для чего проводились последующие замеры после 1 и 7 дней соответственно. Полные результаты замеров приведены в приложении А.

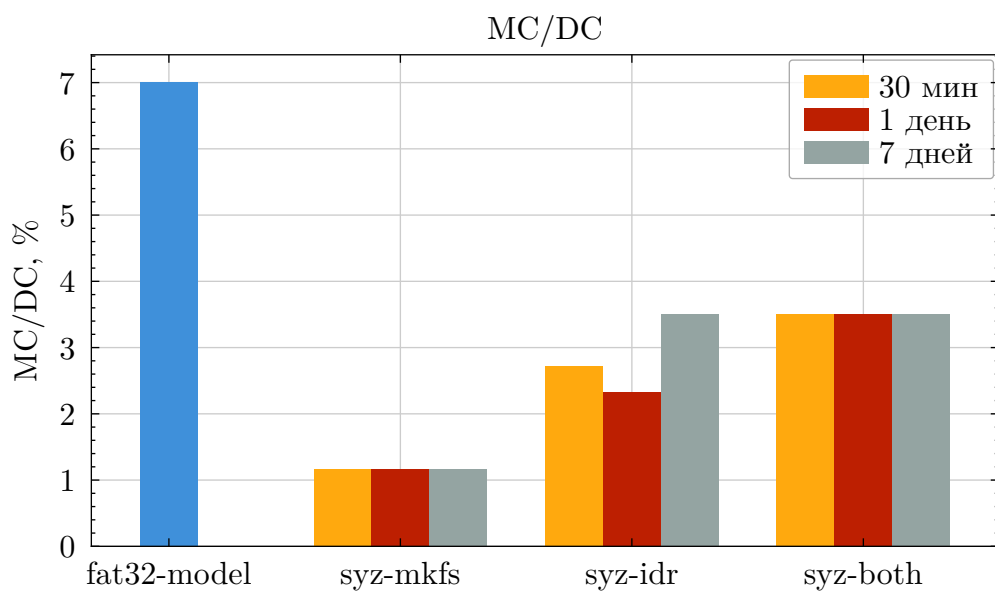
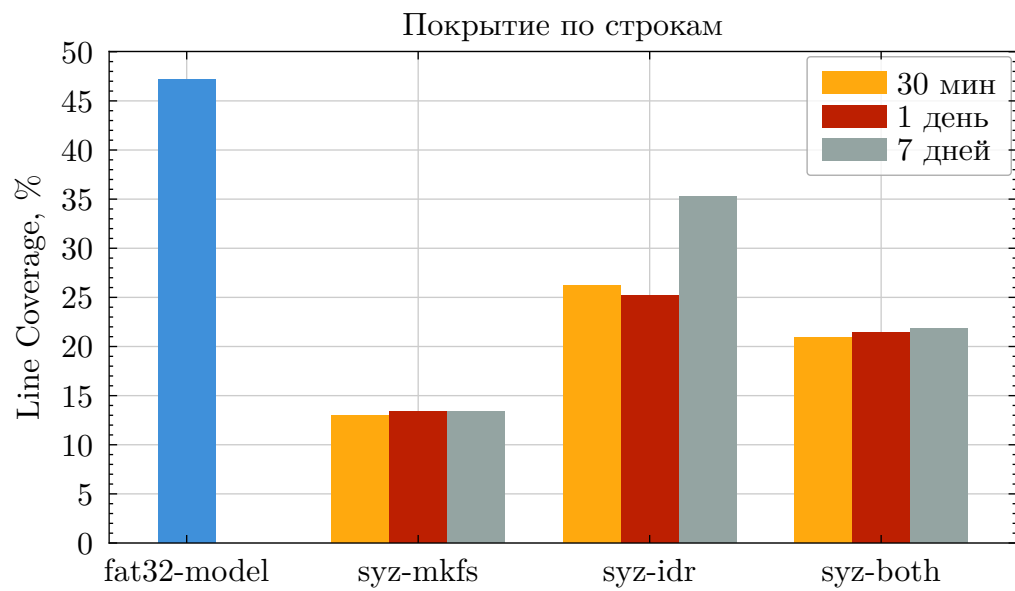
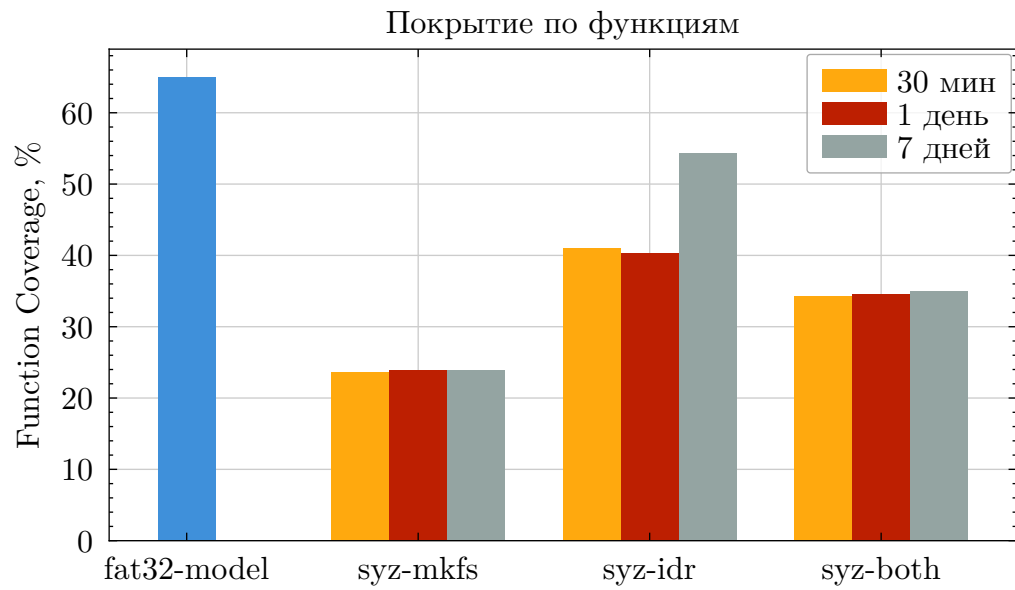


Рис. 4. Графики разных видов покрытия

Как видно из рис. 4, **fat32-model** значительно превосходит **syzkaller** по всем метрикам покрытия. Кроме того, использование сгенерированных разработанным инструментом образов в качестве начальных данных для фаззера также оказало положительное влияние на покрытие.

Заметим, что благодаря модификации фаззера №3, корпус программ, собранных за 7 дней, является надмножеством корпуса, собранного за 1 день, а тот в свою очередь является надмножеством корпуса, собранного за 30 мин. При этом программы в корпус могут записываться фаззером в произвольном порядке, поэтому последовательность программ меньшего корпуса необязательно является префиксом большего. Это может приводить к тому, что из-за порядка исполнения программ покрытие большего корпуса может оказаться меньше, что видно на примере экземпляра **syz-idr** после 30 мин и 1 дня работы соответственно.

Тем не менее, несмотря на количественное превосходство, есть участки кода, покрытые фаззером, но не покрытые **fat32-model**. Для упрощения их обнаружения была использована возможность **llvm-cov** экспортировать отчёты в формат **LCOV**[22] и применён инструмент **lcov-diff**[23] для получения разности покрытия между экземпляром **syz-idr**, работавшим 7 дней, и **fat32-model**.

File	Line Coverage ↕				Function Coverage ↕		
	Rate		Total	Hit	Rate	Total	Hit
cache.c		0.4 %	263	1	5.6 %	18	1
dir.c		0.0 %	1002		0.0 %	33	
fat.h		13.0 %	146	19	7.7 %	207	16
fatent.c		2.2 %	599	13	8.8 %	34	3
file.c		5.2 %	363	19	6.7 %	15	1
inode.c		1.6 %	1252	20	5.5 %	55	3
misc.c		11.2 %	215	24	15.4 %	13	2
namei_msdos.c		0.0 %	507		0.0 %	19	
namei_vfat.c		2.0 %	835	17	4.7 %	43	2
nfs.c		0.0 %	187		0.0 %	11	

Рис. 5. Обзор LCOV покрытого фаззером кода, не покрытого **fat32-model**

Наличие таких участков можно объяснить тремя причинами. Во-первых, как видно на рис. 6, фаззеры лучше покрывают условия и функции, связанные с обработкой ошибок, поскольку значительная часть генерируемых системных вызовов являются невалидными.

```

1847      14 : out_fail:
1848      14 :      iput(fsinfo_inode);
1849      14 :      iput(fat_inode);
1850      14 :      unload_nls(sbi→nls_io);
1851      14 :      unload_nls(sbi→nls_disk);
1852      14 :      fat_reset_iocharset(&sbi→options);
1853      14 :      sb→s_fs_info = NULL;
1854      14 :      kfree(sbi);
1855      14 :      return error;
1856      0 : }

213      : static void fat_write_failed(struct address_space *mapping, loff_t to)
214      133 : {
215      133 :      struct inode *inode = mapping→host;
216      :
217      133 :      if (to > inode→i_size) {
218      133 :          truncate_pagecache(inode, inode→i_size);
219      133 :          fat_truncate_blocks(inode, inode→i_size);
220      133 :      }
221      133 : }
```

Рис. 6. Участки кода, связанные с обработкой ошибок

(■ — покрыто только фаззером, ■ — покрыто обоими или не покрыто фаззером)

Во-вторых, `fat32-model` на данный момент не имеет поддержки длинных имён и умеет работать исключительно со старыми именами формата MS-DOS. Syzkaller не имеет таких ограничений, поэтому он может покрывать участки, связанные с обработкой длинных имён, как видно на рис. 7.

```

618      :      /* build the entry of long file name */
619      2 :      cksum = fat_checksum(msdos_name);
620      :
621      2 :      *nr_slots = usize / 13;
622      4 :      for (ps = slots, i = *nr_slots; i > 0; i--, ps++) {
623      2 :          ps→id = i;
624      2 :          ps→attr = ATTR_EXT;
625      2 :          ps→reserved = 0;
626      2 :          ps→alias_checksum = cksum;
627      2 :          ps→start = 0;
628      2 :          offset = (i - 1) * 13;
629      2 :          fatwchar_to16(ps→name0_4, uname + offset, 5);
630      2 :          fatwchar_to16(ps→name5_10, uname + offset + 5, 6);
631      2 :          fatwchar_to16(ps→name11_12, uname + offset + 11, 2);
632      2 :      }
633      2 :      slots[0].id |= 0x40;
634      2 :      de = (struct msdos_dir_entry *)ps;
```

Рис. 7. Участки кода, связанные с обработкой длинных имён

В-третьих, как видно на рис. 8, при работе фаззера были покрыты участки, связанные с добавлением нового кластера в цепочку при записи в файл. Во время проведения эксперимента при генерации операций с помощью `fat32-model` было установлено ограничение в 256 байт на размер записываемых операцией `Write` данных, что уменьшает вероятность такого события при тестировании. Увеличивая значение этого параметра для отдельных запусков генератора, можно увеличить вероятность добавления нового кластера в цепочку и, как следствие, покрыть эти участки кода.

```

103      : /*
104      :  * fat_chain_add() adds a new cluster to the chain of clusters represented
105      :  * by inode.
106      :  */
107      : int fat_chain_add(struct inode *inode, int new_dclus, int nr_cluster)
108      0 : {
109      0 :     struct super_block *sb = inode->i_sb;
110      0 :     struct msdos_sb_info *sbi = MSDOS_SB(sb);
111      0 :     int ret, new_fclus, last;
112      :
113      :     /*
114      :     * We must locate the last cluster of the file to add this new
115      :     * one (new_dclus) to the end of the link list (the FAT).
116      :     */
117      0 :     last = new_fclus = 0;
118      0 :     if (MSDOS_I(inode)->i_start) {
119      1567 :         int fclus, dclus;
120      :
121      1567 :         ret = fat_get_cluster(inode, FAT_ENT_EOF, &fclus, &dclus);
122      1567 :         if (ret < 0)
123      0 :             return ret;
124      1567 :         new_fclus = fclus + 1;
125      1567 :         last = dclus;
126      1567 :     }
127      :
128      :     /* add new one to the last of the cluster chain */
129      0 :     if (last) {
130      1567 :         struct fat_entry fatent;
131      :
132      1567 :         fatent_init(&fatent);
133      1567 :         ret = fat_ent_read(inode, &fatent, last);
134      1567 :         if (ret ≥ 0) {
135      1567 :             int wait = inode_needs_sync(inode);
136      1567 :             ret = fat_ent_write(inode, &fatent, new_dclus, wait);
137      1567 :             fatent_brelse(&fatent);
138      1567 :         }
139      1567 :         if (ret < 0)

```

Рис. 8. Участки кода, связанные с добавлением нового кластера в цепочку

5.3 Обобщение модели

В рамках данной работы была разработана библиотека `fs-model`[10], позволяющая построить аналогичную инфраструктуру генерации семантически корректных операций для других файловых систем, и реализована модель ФС FAT32, выраженная средствами этой библиотеки. Из-за вышеупомянутых ограничений DepTyCheck на момент написания статьи генерация программ не поддерживается.

6 Заключение

В данной работе была создана формальная спецификация с использованием зависимых типов для семантически корректных операций над файловой системой FAT32. На её основе был успешно разработан инструмент, генерирующий программы на языке Си, производящие эти операции и проверяющие их результат. С помощью утилиты сбора покрытия `llvm-cov` было проведено сравнение разработанного инструмента с фаззером `syzkaller`, которое показало значительное превосходство `fat32-model` по количеству покрытого кода как в сравнимые промежутки времени, так и при более продолжительной работе фаззера. Использование сгенерированных разработанным инструментом образов в качестве начальных данных для фаззинга также дало положительный результат. При качественном сравнении покрытия были выявлены участки кода, где `fat32-model` проявил себя хуже, чем `syzkaller`, и проанализированы возможные причины этого. Также в рамках работы было исследовано обобщение разработанного метода и создана библиотека `fs-model` для упрощения разработки подобных инструментов для других файловых систем.

Возможные направления для дальнейшей работы включают в себя:

- добавление поддержки новых операций;
- доработка модели ФС FAT32: поддержка длинных имён, работа с временными метками и т. д.;
- использование разработанной обобщённой модели для реализации генераторов для других файловых систем;
- обобщение вычисления значений результатов операций и добавление поддержки других форматов вывода операций;
- расширение обобщённой модели новыми видами файлов, например, устройствами и символьными ссылками;
- моделирование жёстких ссылок.

Список литературы

1. Brady E. Idris, a general-purpose dependently typed programming language: Design and implementation // Journal of Functional Programming. 2013. Т. 23, № 5. Сс. 552–593.
2. Brady E. Idris 2: Quantitative Type Theory in Practice: 2104.0048. 2021.
3. GitHub-репозиторий Idris 2 [Электронный ресурс]. URL: <https://github.com/idris-lang/Idris2>.
4. Dependent type theory [Электронный ресурс]. URL: <https://ncatlab.org/nlab/show/dependent+type+theory>.
5. Buzdalov D. Dependent Types to Push Corners of the Property-based Testing (Extended Abstract) // ICFP. 2024.
6. GitHub-репозиторий DepTyCheck [Электронный ресурс]. URL: <https://github.com/buzden/deptycheck>.
7. Christiansen D., Brady E. Elaborator reflection: extending Idris in Idris // Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming. Association for Computing Machinery, 2016. Сс. 284–197.
8. Денисьев И.Д. Автоматизация тестирования драйвера FAT32 при помощи генерации корректных образов файловой системы с использованием её формальной спецификации на основе зависимых типов: Курсовая работа. 2025.
9. GitHub-репозиторий fat32-model [Электронный ресурс]. URL: <https://github.com/DepTyCheck/fat32-model>.
10. GitHub-репозиторий fs-model [Электронный ресурс]. URL: <https://github.com/DepTyCheck/fs-model>.
11. GitHub-репозиторий syzkaller [Электронный ресурс]. URL: <https://github.com/google/syzkaller>.
12. Генерация начальных образов в коде syzkaller [Электронный ресурс]. URL: <https://github.com/google/syzkaller/blob/master/tools/syz-imagegen/imagegen.go>.
13. Xu W. и др. Fuzzing File Systems via Two-Dimensional Input Space Exploration // 2019 IEEE Symposium on Security and Privacy (SP). 2019. Сс. 818–834.
14. Purdila O., Grijincu L.A., Tapus N. LKL: The Linux kernel library // 9th RoEduNet IEEE International Conference. 2010. Сс. 328–333.
15. Mehta M.P. Formalising Filesystems in the ACL2 Theorem Prover: an Application to FAT32 // Electronic Proceedings in Theoretical Computer Science. Open Publishing Association, 2018. Т. 280. Сс. 18–29.
16. Microsoft Extensible Firmware Initiative FAT32 File System Specification, FAT: General Overview of On-Disk Format.

17. KCOV: code coverage for fuzzing [Электронный ресурс]. URL: <https://www.kernel.org/doc/html/latest/dev-tools/kcov.html>.
18. Using gcov with the Linux kernel [Электронный ресурс]. URL: <https://www.kernel.org/doc/html/latest/dev-tools/gcov.html>.
19. Zhang W. и др. An Open-Source Structural Coverage Tool for DO-178C Compliance // AIAA DATC/IEEE 44th Digital Avionics Systems Conference (DASC '25). 2025.
20. Source-based Code Coverage [Электронный ресурс]. URL: <https://clang.llvm.org/docs/SourceBasedCodeCoverage.html>.
21. GitHub-репозиторий модифицированного фаззера syzkaller [Электронный ресурс]. URL: <https://github.com/elkcl/syzkaller/tree/bachelor-thesis>.
22. GitHub-репозиторий LCOV [Электронный ресурс]. URL: <https://github.com/linux-test-project/lcov>.
23. GitHub-репозиторий lcov-diff [Электронный ресурс]. URL: <https://github.com/capgelka/lcov-diff>.

Приложение А

Результаты замеров покрытия фаззера syzkaller

syz-mkfs	Filename	Function Coverage	Line Coverage	MC/DC
	cache.c	55.56% (10/18)	41.22% (108/262)	0.00% (0/6)
	dir.c	15.62% (5/32)	5.82% (58/997)	4.69% (3/64)
	fat.h	56.52% (13/23)	41.38% (60/145)	0.00% (0/4)
	fatent.c	20.59% (7/34)	9.68% (58/599)	0.00% (0/21)
	file.c	0.00% (0/15)	0.00% (0/361)	0.00% (0/19)
	inode.c	36.36% (20/55)	28.75% (360/1252)	0.00% (0/38)
	misc.c	15.38% (2/13)	13.40% (28/209)	0.00% (0/10)
	namei_msdos.c	0.00% (0/19)	0.00% (0/507)	0.00% (0/49)
	namei_vfat.c	11.63% (5/43)	2.53% (21/830)	0.00% (0/37)
	nfs.c	0.00% (0/11)	0.00% (0/185)	0.00% (0/9)
	Totals	23.57% (62/263)	12.96% (693/5347)	1.17% (3/257)
syz-idr	Filename	Function Coverage	Line Coverage	MC/DC
	cache.c	55.56% (10/18)	41.22% (108/262)	0.00% (0/6)
	dir.c	50.00% (16/32)	31.49% (314/997)	9.38% (6/64)
	fat.h	78.26% (18/23)	64.83% (94/145)	0.00% (0/4)
	fatent.c	20.59% (7/34)	9.68% (58/599)	0.00% (0/21)
	file.c	6.67% (1/15)	1.39% (5/361)	0.00% (0/19)
	inode.c	50.91% (28/55)	36.82% (461/1252)	0.00% (0/38)
	misc.c	30.77% (4/13)	23.44% (49/209)	0.00% (0/10)
	namei_msdos.c	0.00% (0/19)	0.00% (0/507)	0.00% (0/49)
	namei_vfat.c	55.81% (24/43)	37.47% (311/830)	2.70% (1/37)
	nfs.c	0.00% (0/11)	0.00% (0/185)	0.00% (0/9)
	Totals	41.06% (108/263)	26.18% (1400/5347)	2.72% (7/257)
syz-both	Filename	Function Coverage	Line Coverage	MC/DC
	cache.c	55.56% (10/18)	41.22% (108/262)	0.00% (0/6)
	dir.c	43.75% (14/32)	26.78% (267/997)	14.06% (9/64)
	fat.h	69.57% (16/23)	59.31% (86/145)	0.00% (0/4)
	fatent.c	20.59% (7/34)	9.68% (58/599)	0.00% (0/21)
	file.c	6.67% (1/15)	2.49% (9/361)	0.00% (0/19)
	inode.c	49.09% (27/55)	37.46% (469/1252)	0.00% (0/38)
	misc.c	15.38% (2/13)	14.35% (30/209)	0.00% (0/10)
	namei_msdos.c	0.00% (0/19)	0.00% (0/507)	0.00% (0/49)
	namei_vfat.c	30.23% (13/43)	10.84% (90/830)	0.00% (0/37)
	nfs.c	0.00% (0/11)	0.00% (0/185)	0.00% (0/9)
	Totals	34.22% (90/263)	20.89% (1117/5347)	3.50% (9/257)

Таблица 1. Статистика покрытия после 30 мин работы фаззеров

syz-mkfs	Filename	Function Coverage	Line Coverage	MC/DC
	cache.c	55.56% (10/18)	41.22% (108/262)	0.00% (0/6)
	dir.c	15.62% (5/32)	5.82% (58/997)	4.69% (3/64)
	fat.h	56.52% (13/23)	41.38% (60/145)	0.00% (0/4)
	fatent.c	20.59% (7/34)	9.68% (58/599)	0.00% (0/21)
	file.c	0.00% (0/15)	0.00% (0/361)	0.00% (0/19)
	inode.c	36.36% (20/55)	29.71% (372/1252)	0.00% (0/38)
	misc.c	23.08% (3/13)	17.70% (37/209)	0.00% (0/10)
	namei_msdos.c	0.00% (0/19)	0.00% (0/507)	0.00% (0/49)
	namei_vfat.c	11.63% (5/43)	2.53% (21/830)	0.00% (0/37)
	nfs.c	0.00% (0/11)	0.00% (0/185)	0.00% (0/9)
	Totals	23.95% (63/263)	13.35% (714/5347)	1.17% (3/257)
syz-idr	Filename	Function Coverage	Line Coverage	MC/DC
	cache.c	55.56% (10/18)	41.22% (108/262)	0.00% (0/6)
	dir.c	46.88% (15/32)	26.78% (267/997)	7.81% (5/64)
	fat.h	73.91% (17/23)	62.07% (90/145)	0.00% (0/4)
	fatent.c	20.59% (7/34)	9.68% (58/599)	0.00% (0/21)
	file.c	6.67% (1/15)	1.39% (5/361)	0.00% (0/19)
	inode.c	50.91% (28/55)	36.82% (461/1252)	0.00% (0/38)
	misc.c	30.77% (4/13)	23.44% (49/209)	0.00% (0/10)
	namei_msdos.c	0.00% (0/19)	0.00% (0/507)	0.00% (0/49)
	namei_vfat.c	55.81% (24/43)	37.47% (311/830)	2.70% (1/37)
	nfs.c	0.00% (0/11)	0.00% (0/185)	0.00% (0/9)
	Totals	40.30% (106/263)	25.23% (1349/5347)	2.33% (6/257)
syz-both	Filename	Function Coverage	Line Coverage	MC/DC
	cache.c	55.56% (10/18)	41.22% (108/262)	0.00% (0/6)
	dir.c	46.88% (15/32)	28.18% (281/997)	14.06% (9/64)
	fat.h	69.57% (16/23)	60.69% (88/145)	0.00% (0/4)
	fatent.c	20.59% (7/34)	9.68% (58/599)	0.00% (0/21)
	file.c	6.67% (1/15)	2.49% (9/361)	0.00% (0/19)
	inode.c	49.09% (27/55)	38.50% (482/1252)	0.00% (0/38)
	misc.c	15.38% (2/13)	14.35% (30/209)	0.00% (0/10)
	namei_msdos.c	0.00% (0/19)	0.00% (0/507)	0.00% (0/49)
	namei_vfat.c	30.23% (13/43)	10.84% (90/830)	0.00% (0/37)
	nfs.c	0.00% (0/11)	0.00% (0/185)	0.00% (0/9)
	Totals	34.60% (91/263)	21.43% (1146/5347)	3.50% (9/257)

Таблица 2. Статистика покрытия после 1 дня работы фаззеров

syz-mkfs	Filename	Function Coverage	Line Coverage	MC/DC
	cache.c	55.56% (10/18)	41.22% (108/262)	0.00% (0/6)
	dir.c	15.62% (5/32)	5.82% (58/997)	4.69% (3/64)
	fat.h	56.52% (13/23)	41.38% (60/145)	0.00% (0/4)
	fatent.c	20.59% (7/34)	9.68% (58/599)	0.00% (0/21)
	file.c	0.00% (0/15)	0.00% (0/361)	0.00% (0/19)
	inode.c	36.36% (20/55)	29.71% (372/1252)	0.00% (0/38)
	misc.c	23.08% (3/13)	17.70% (37/209)	0.00% (0/10)
	namei_msdos.c	0.00% (0/19)	0.00% (0/507)	0.00% (0/49)
	namei_vfat.c	11.63% (5/43)	2.53% (21/830)	0.00% (0/37)
	nfs.c	0.00% (0/11)	0.00% (0/185)	0.00% (0/9)
	Totals	23.95% (63/263)	13.35% (714/5347)	1.17% (3/257)
syz-idr	Filename	Function Coverage	Line Coverage	MC/DC
	cache.c	88.89% (16/18)	75.19% (197/262)	16.67% (1/6)
	dir.c	46.88% (15/32)	26.78% (267/997)	7.81% (5/64)
	fat.h	86.96% (20/23)	72.41% (105/145)	0.00% (0/4)
	fatent.c	55.88% (19/34)	38.73% (232/599)	9.52% (2/21)
	file.c	20.00% (3/15)	14.13% (51/361)	0.00% (0/19)
	inode.c	65.45% (36/55)	46.81% (586/1252)	0.00% (0/38)
	misc.c	76.92% (10/13)	65.07% (136/209)	0.00% (0/10)
	namei_msdos.c	0.00% (0/19)	0.00% (0/507)	0.00% (0/49)
	namei_vfat.c	55.81% (24/43)	37.47% (311/830)	2.70% (1/37)
	nfs.c	0.00% (0/11)	0.00% (0/185)	0.00% (0/9)
	Totals	54.37% (143/263)	35.25% (1885/5347)	3.50% (9/257)
syz-both	Filename	Function Coverage	Line Coverage	MC/DC
	cache.c	55.56% (10/18)	41.22% (108/262)	0.00% (0/6)
	dir.c	46.88% (15/32)	28.18% (281/997)	14.06% (9/64)
	fat.h	69.57% (16/23)	60.69% (88/145)	0.00% (0/4)
	fatent.c	20.59% (7/34)	9.68% (58/599)	0.00% (0/21)
	file.c	6.67% (1/15)	2.49% (9/361)	0.00% (0/19)
	inode.c	49.09% (27/55)	39.46% (494/1252)	0.00% (0/38)
	misc.c	23.08% (3/13)	18.66% (39/209)	0.00% (0/10)
	namei_msdos.c	0.00% (0/19)	0.00% (0/507)	0.00% (0/49)
	namei_vfat.c	30.23% (13/43)	10.84% (90/830)	0.00% (0/37)
	nfs.c	0.00% (0/11)	0.00% (0/185)	0.00% (0/9)
	Totals	34.98% (92/263)	21.83% (1167/5347)	3.50% (9/257)

Таблица 3. Статистика покрытия после 7 дней работы фаззеров